

Akhbar El-Youm Academy

Faculty of Computer Science & Information Technology



AI Accountant

An Intelligent Financial Management System

*A Graduation Project Submitted in Partial Fulfillment of the Requirements for the Bachelor of
Computer Science Degree*

ACADEMIC YEAR

2025 / 2026

PROJECT TYPE

Graduation Project

DEPARTMENT

Computer Science

DOCUMENT DATE

June 2026

React 19

Express 5

MongoDB / Mongoose

TailwindCSS 4

Groq SDK

Google Generative AI

JWT Auth

ExcelJS

Abstract

This report presents the design, architecture, and implementation of **AI Accountant**, an intelligent financial management system developed as a graduation project at Akhbar El-Youm Academy, Faculty of Computer Science, in partial fulfillment of the requirements for the Bachelor of Computer Science degree. The system addresses the financial management challenges commonly encountered by small and medium-sized enterprises (SMEs), where affordable, intelligent accounting tools with real-time analytical capabilities remain scarce.

The proposed system delivers a comprehensive suite of accounting functionalities, including real-time Key Performance Indicator (KPI) dashboards, automated transaction classification, AI-driven spreadsheet generation, natural-language financial querying via a conversational chatbot interface, and a versioned in-browser Excel editor. Two AI providers are leveraged: the Groq inference engine running the LLaMA 3.3-70B model for conversational and generative tasks, and Google Gemini for multi-modal document analysis and automated transaction extraction from uploaded financial files.

The system adopts a client-server architecture comprising a React 19 Single-Page Application (SPA) on the frontend and a Node.js/Express 5 RESTful API server on the backend, with data persisted in a MongoDB database via the Mongoose ODM. A tiered subscription model governs feature access, and a versioned public REST API enables programmatic integration by third-party developers. Security is enforced through stateless JWT authentication, bcrypt-hashed credentials, Helmet HTTP security headers, and scope-based API key authorization, all conforming to OWASP Top Ten security guidelines.

The implemented system demonstrates the technical viability of integrating generative AI within a traditional enterprise accounting workflow, substantially reducing manual data-entry overhead and enhancing financial visibility for business operators.

Keywords: Artificial Intelligence, Financial Management, Large Language Models, RESTful API, Single-Page Application, React, Node.js, MongoDB, Groq, LLaMA, Google Gemini, Generative AI, SME Accounting, Natural Language Processing, JWT Authentication.

Table of Contents

Abstract

3

1. Project Overview

4

1.1 Problem Statement & Research Objectives

4

1.2 Technology Stack

4

1.3 Background & Related Work

5

2. System Architecture

5

2.1 High-Level Architecture

5

2.2 Directory Structure

5

3. Backend

6

3.1 Server Setup & Middleware

6

3.2 Data Models

7

3.3 Authentication System

8

3.4 Transactions Engine

9

3.5 Dashboard & KPI Analytics

9

3.6 AI Integrations

10

3.7 Excel Processing

11

3.8 Admin Panel

11

4. REST API Reference

12

4.1 Auth Endpoints

12

4.2 Transaction Endpoints

12

4.3 Dashboard Endpoints	13
4.4 AI Endpoints	13
4.5 Subscription Endpoints	14
4.6 Public v1 API (API Key Auth)	14
5. Frontend	15
5.1 Structure & Routing	15
5.2 Pages & Components	15
5.3 State Management	16
6. Subscription & Billing	17
7. Security	18
8. Developer Guide	19
8.1 Local Setup	19
8.2 Environment Variables	19
8.3 Developer API Keys	20
9. Deployment	21
10. Email Service	22
11. Conclusion & Future Work	23
11.1 Conclusion	23
11.2 Limitations	24
11.3 Future Work	24
12. References & Bibliography	25



1. Project Overview

AI Accountant is a full-stack, intelligent financial management system developed to address the accounting and analytical requirements of small and medium-sized enterprises (SMEs). The system synthesizes conventional bookkeeping methodologies with state-of-the-art artificial intelligence capabilities, encompassing automated transaction classification, AI-driven spreadsheet generation, real-time financial Key Performance Indicator (KPI) dashboards, and a natural-language conversational financial advisory interface powered by large language models (LLMs).

This system was conceived and implemented as a graduation capstone project at Akhbar El-Youm Academy, Faculty of Computer Science, during the academic year 2025/2026. It demonstrates the practical application of modern full-stack web engineering principles, distributed system design patterns, and the integration of generative AI language models within a production-grade software architecture.

1.1 Problem Statement & Research Objectives

Small and medium-sized enterprises frequently encounter significant challenges in managing financial operations efficiently. Traditional accounting software solutions are either prohibitively expensive, require dedicated accounting personnel, or lack the intelligence necessary to deliver actionable business insights in real time. The recent proliferation of large language models (LLMs) and generative AI presents a compelling opportunity to augment conventional accounting tools with natural-language interfaces, automated data extraction, and intelligent financial advisory capabilities.

The following primary objectives guided the design and implementation of this system:

**Financial Visibility**

Provide business owners with real-time KPI dashboards — revenue, expenses, profit margin, cash flow, burn rate, and tax estimates — with month-over-month trend comparisons.

**AI-Powered Analysis**

Integrate Groq (LLaMA 3) and Google Gemini AI to enable natural-language financial queries, automatic transaction categorization, chart generation, and spreadsheet creation.

**Smart Data Ingestion**

Accept Excel (.xlsx, .xls) and PDF financial files, automatically extract and analyze transaction data using AI, and batch-import thousands of records via a background job queue.

**Developer API**

Expose a versioned public REST API (v1) secured by API keys, allowing third-party integrations to read/write transactions and retrieve dashboard KPIs programmatically.

1.2 Technology Stack

Layer	Technology	Purpose
Frontend	React 19, Vite 7, TailwindCSS 4	SPA with fast HMR development
Frontend	React Router v7, Zustand, TanStack Query	Routing, global state, server-state caching
Frontend	Recharts, Flowbite React	Data visualization, UI component library
Backend	Node.js, Express 5	REST API server
Backend	Mongoose 8, MongoDB	Primary database (ODM)
Backend	JWT (jsonwebtoken)	Stateless authentication
Backend	Helmet, express-rate-limit	Security headers and rate limiting
AI	Groq SDK (LLaMA 3.3-70b)	Chatbot, chart generation, Excel generation
AI	Google Generative AI (Gemini)	Document parsing, AI analysis
Files	ExcelJS, Multer, pdf-parse	Excel/PDF file handling and parsing
Email	Nodemailer, Resend	Transactional emails (password reset)
Validation	Zod, express-validator	Input validation (FE + BE)

1.3 Background & Related Work

The application of artificial intelligence to financial management software has attracted considerable research and industrial interest in recent years. Established commercial platforms such as QuickBooks Online, Xero, and Wave Accounting offer robust bookkeeping functionality but provide limited AI-driven intelligence beyond rule-based automation heuristics. Academic research has investigated LLM-based document information extraction, natural-language interfaces for structured database querying, and supervised machine-learning models for transaction categorization — capabilities that this project unifies within a single cohesive system.

This project builds upon and synthesizes prior work in four principal areas: (i) **RESTful API design** adhering to Fielding’s representational state transfer (REST) architectural constraints for stateless client-server communication; (ii) **LLM integration** leveraging the Groq inference API (LLaMA 3.3-70B) for text generation tasks and the Google Gemini API for multi-modal document understanding; (iii) **modern Single-Page Application (SPA) architecture** employing React 19 with optimistic server-state management via TanStack Query; and (iv) **security-first backend design** conforming to OWASP Top Ten security guidelines including input validation, rate limiting, and zero-knowledge credential storage.



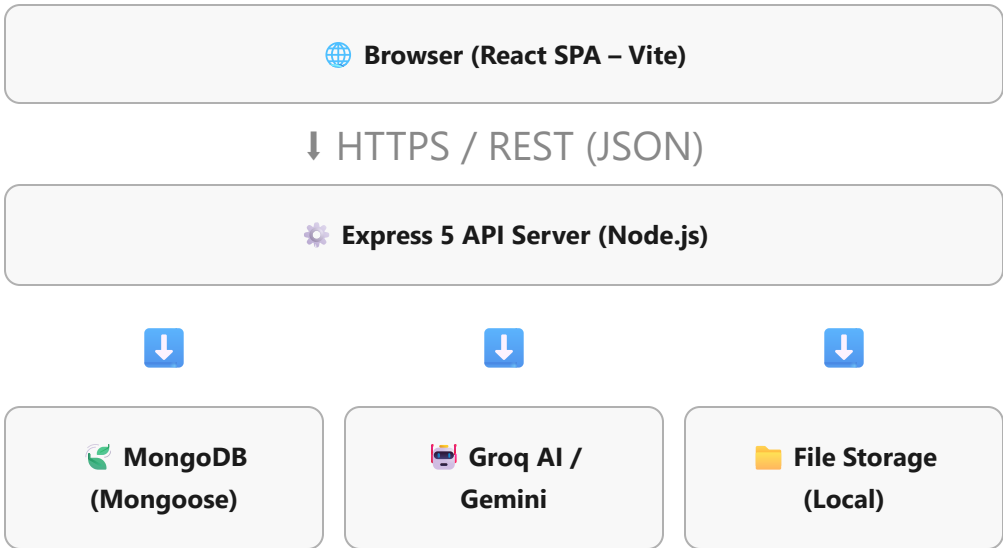
Distinguishing Contribution

Unlike existing SME accounting platforms, this system integrates two complementary AI providers within a unified architecture: Groq's LLaMA 3.3-70B for conversational dialogue and generative spreadsheet creation, and Google Gemini for multi-modal financial document parsing. This combination enables end-to-end AI-assisted accounting workflows spanning document ingestion, automated transaction classification, interactive natural-language querying, and on-demand report generation.



2. System Architecture

2.1 High-Level Architecture



Dual Data-Store Pattern

The backend automatically falls back to a local file-based store (JSON files on disk) when MongoDB is unavailable or when the user ID is not a valid MongoDB ObjectId. This allows the app to function fully in development without a database connection.

2.2 Directory Structure

```
aiAccountant/
├── backend/ - Node.js / Express API server
│   ├── server.js - Entry point, route mounting
│   └── config/
```

```
| | └─ constants.js - App-wide constants, subscription plans
| | └─ controllers/ - Request handlers (business logic)
| | └─ middleware/ - Auth, rate-limit, validation, checkLimit
| | └─ models/ - Mongoose schemas
| | └─ routes/ - Express route definitions
| | └─ services/ - Business services, local stores, email
| | └─ validators/ - Zod / express-validator schemas
| | └─ uploads/ - File uploads + usage JSON store
| └─ frontend/ - React 19 / Vite SPA
|   └─ src/
|     └─ App.jsx - Root component & router
|     └─ Component/ - Auth forms, chatbot, excel tools
|     └─ components/ - Dashboard, Transactions, ExcelEditor
|     └─ pages/ - Landing, Pricing, Admin, Developer
|     └─ Context/ - Auth, Theme, Subscription contexts
|     └─ stores/ - Zustand stores
|     └─ hooks/ - Custom React hooks
|     └─ services/ - API call abstractions
|     └─ layouts/ - Dashboard, Transactions layouts
| └─ docs/ - Architecture docs & wireframes
```



3. Backend

3.1 Server Setup & Middleware

The Express 5 server is the entry point at `backend/server.js`. It configures a layered middleware pipeline before mounting route modules.

Middleware	Purpose	Config
<code>helmet</code>	Sets secure HTTP headers (CSP, HSTS, X-Frame-Options, etc.)	Default config
<code>cors</code>	Restricts origins; in production only allows <code>CLIENT_URL</code> env list	Credentials: true, All methods
<code>express.json</code>	Body parser for JSON payloads	25 MB limit
Request Logger	Logs request ID, method, route, status, latency, response bytes	Custom middleware with <code>hrtime</code>
<code>apiLimiter</code>	Global rate limiter on all <code>/api/*</code> routes	2000 req / 15 min (prod only)
<code>authLimiter</code>	Stricter rate limiter on auth endpoints	10 attempts / 15 min (prod only)
Error Handler	Catches all unhandled errors, returns structured JSON	Hides stack traces in production

Health Endpoint

The `GET /health` endpoint returns live database connectivity status, uptime, memory usage, and MongoDB/Redis health.

3.2 Data Models

User Model

Field	Type	Notes
name	String	Required
email	String	Required, unique
password	String	bcrypt hashed (10 rounds)
businessType	Enum	<code>retail</code> <code>wholesale</code> <code>service</code>
resetPasswordTokenHash	String	SHA-256 hash of reset token (never stored plaintext)
resetPasswordExpiresAt	Date	1-hour expiry
passwordChangedAt	Date	Set on password change
createdAt / updatedAt	Date	Auto-managed by Mongoose timestamps

Transaction Model

Field	Type	Notes
date	Date	Required
desc	String	Required — transaction description
amount	Number	Required — always stored as positive; type field determines direction
currency	String	Default: USD
category	String	Required
subcategory	String	Optional
type	Enum	<code>income</code> <code>expense</code> <code>transfer</code>
vendor	String	Payee name
paymentMethod	Enum	<code>cash</code> / <code>credit_card</code> / <code>debit_card</code> / <code>bank_transfer</code> / <code>check</code> / <code>other</code>
status	Enum	<code>pending</code> / <code>approved</code> / <code>rejected</code> / <code>needs_review</code> / <code>reconciled</code> / <code>flagged</code>
sourceFile	Object	Filename, original name, type (excel/pdf), upload date
aiAnalysis	Object	confidence score, extractedText, suggestedCategory, processingNotes
isReconciled	Boolean	Reconciliation flag

Subscription Model

Field	Type	Notes
user	ObjectId → User	One-to-one relationship
plan	Enum	<code>free</code> <code>pro</code> <code>business</code>
status	Enum	<code>active</code> / <code>cancelled</code> / <code>past_due</code>
billingCycle	Enum	<code>monthly</code> / <code>annual</code>
cancelAtPeriodEnd	Boolean	Graceful cancellation flag
paymentMethod	Object	Stores last4 / brand / expiry (no raw card numbers)
invoices	Array	Invoice history (amount, status, plan, billingCycle, paidAt)

ApiKey Model

Developer API keys follow a zero-knowledge storage pattern:

- Wire format: `ak_<32 random hex bytes>` (68 chars total)
- Only the **SHA-256 hash** of the key is stored in the database

- The first 10 characters (`keyPrefix`) are stored as a safe display hint
- Scopes control access: `transactions:read`, `transactions:write`, `dashboard:read`, `excel:read`, `excel:write`, `chatbot:read`

3.3 Authentication System

Authentication uses **stateless JWT tokens** with a 72-hour expiry. The system supports both MongoDB-backed and local file-based user storage.

Registration Flow

1. Client POSTs `name`, `email`, `password`, `businessType`
2. Input validated by express-validator rules
3. Password hashed with bcrypt (10 salt rounds)
4. User created in MongoDB or local JSON store (dev fallback)
5. JWT signed and returned to client

Login Flow

1. Client POSTs `email` + `password`
2. User fetched from store; bcrypt compare performed
3. On success: JWT (72h) returned; client stores in memory / localStorage
4. All subsequent requests attach `Authorization: Bearer <token>`

Password Reset Flow

1. User POSTs email to `/api/auth/forgot-password`
2. Crypto random token generated; SHA-256 hash stored in DB with 1h expiry
3. Plain token sent via email link (Nodemailer / Resend)
4. User visits reset link → POSTs new password + token to `/api/auth/reset-password`
5. Token hash verified; new password bcrypt-hashed and saved

Admin Detection

Admin status is determined at token-sign time by checking the user's email against the `ADMIN_EMAILS` environment variable (comma-separated list). The `isAdmin` flag is embedded in the JWT payload.

3.4 Transactions Engine

The transaction system supports CRUD operations, bulk operations, file uploads, and a streaming background import job queue.

File Upload & AI Analysis

1. Multer accepts Excel (.xlsx, .xls) and PDF files (max 10 MB)
2. ExcelJS parses Excel sheets; pdf-parse extracts PDF text
3. Google Gemini AI analyzes extracted content to auto-classify transactions (category, type, amount, date)

4. Results returned as structured JSON for user review before saving

Background Import Jobs

Large imports use a streaming job queue to avoid HTTP timeouts:

1. **Create Job:** POST `/api/transactions/import-jobs` → returns `jobId`
2. **Stream Status:** GET `/api/transactions/import-jobs/:id/stream` (SSE)
3. **Add Chunks:** POST `/api/transactions/import-jobs/:id/chunks`
4. **Finalize:** POST `/api/transactions/import-jobs/:id/finalize`
5. **Retry / Cancel:** POST `... /retry` or `... /cancel`

3.5 Dashboard & KPI Analytics

The `calculateDashboard` function computes financial KPIs from raw transaction arrays without aggregation queries, making it compatible with both the MongoDB store and local JSON store.

KPI	Calculation
Total Revenue	Sum of all <code>income</code> transactions
Total Expenses	Sum of all <code>expense</code> transactions
Net Profit	Revenue – Expenses
Profit Margin	$(\text{Net Profit} / \text{Revenue}) \times 100\%$
Revenue Growth	$(\text{Current Month Revenue} - \text{Previous Month Revenue}) / \text{Previous Month Revenue} \times 100\%$
Burn Rate	Current month expenses (or average monthly expenses)
Runway	$\text{Net Profit} / \text{Burn Rate (months)}$
Tax Estimate	Current month profit $\times 22\%$
Pending Count	Count of transactions with <code>pending</code> or <code>needs_review</code> status
Monthly Breakdown	Revenue, expenses, profit, cash flow per calendar month

3.6 AI Integrations

The platform uses two AI providers through separate controllers:

Groq (LLaMA 3)

Model: llama-3.3-70b-versatile

Used for:

- AI Chatbot — natural language financial Q&A
- AI Chart Generator — generates chart configs from text prompts
- AI Excel Generator — creates full Excel spreadsheets from text descriptions

Google Gemini

Model: gemini-pro / gemini-flash

Used for:

- Document parsing (PDF / Excel transaction extraction)
- AI financial analysis and categorization
- Supplementary chart and insight generation

AI Chatbot Architecture

The chatbot controller queries the user's transactions (both MongoDB and local store compatible) and builds a structured context for the AI. It supports:

- Grouping transactions by `category`, `month`, or `type`
- Sorting by `amount`, `date`, or `count`
- Date range filtering
- Multi-turn conversation history via MongoDB `Chat` model
- Usage tracking (per-user monthly AI chat message count)

AI Excel Generator

Accepts a natural-language description and uses Groq to produce a custom delimited text format that is then parsed and rendered into a real Excel file using ExcelJS. Supports:

- Custom column widths and row heights
- Cell fill colors and font formatting (bold, size, color)
- Formula cells
- Merged cell ranges
- Multiple named sheets

3.7 Excel Processing

The Excel Editor module (`excelEditorController`) allows users to upload, view, and edit Excel files in-browser with version history:

- Files stored in `uploads/excel/` with user-scoped access
- ExcelJS used for reading/writing .xlsx files
- FileVersion model tracks change history
- Reconciler service cross-references bank statements with transactions

3.8 Admin Panel

The admin controller provides platform-level management accessible only to users whose email is listed in `ADMIN_EMAILS`:

Platform Stats

Total user count, plan distribution (free/pro/business), new signups (last 7 days), monthly revenue estimate, blog post counts

User Management

List all users, view subscription status, manage account states. Admin emails are excluded from user-facing stats.

CMS

Blog post CRUD (title, content, category, status), landing page content, navigation links, and pricing tier management

Tier Management

Dynamically update subscription plan names, prices, features, and usage limits without code changes



4. REST API Reference



Authentication

All protected endpoints require the header `Authorization: Bearer <JWT_TOKEN>`. The Public v1 API uses `X-API-Key: <API_KEY>` header instead.

4.1 Auth Endpoints

Base: `/api/auth`

POST `/register`

Register a new user. Body: `name`, `email`, `password`, `businessType`. Returns JWT + user object.

POST `/login`

Authenticate. Body: `email`, `password`. Returns JWT + user object.

POST `/forgot-password`

Trigger password reset email. Body: `email`. Rate limited (10/15 min).

POST `/reset-password`

Reset password using token. Body: `token`, `password`. Token expires in 1 hour.

4.2 Transaction Endpoints

Base: `/api/transactions`

POST	/upload	Upload Excel or PDF file for AI analysis. Multipart form-data with <code>file</code> field. Returns analyzed transaction array.
POST	/import-jobs	Create a background import job. Returns <code>jobId</code> .
GET	/import-jobs/:jobId	Get import job status and progress.
GET	/import-jobs/:jobId/stream	Server-Sent Events stream for real-time import progress.
POST	/import-jobs/:jobId/chunks	Add a batch of transactions to an import job.
POST	/import-jobs/:jobId/finalize	Commit the import job and save all transactions.
POST	/import-jobs/:jobId/retry	Retry failed chunks in an import job.
POST	/import-jobs/:jobId/cancel	Cancel a pending import job.
POST	/	Create a single manual transaction. Limit-checked against subscription plan.
GET	/	List transactions. Query params: <code>limit</code> , <code>page</code> , <code>cursor</code> , <code>category</code> , <code>status</code> , <code>type</code> , <code>search</code> , <code>dateFrom</code> , <code>dateTo</code> , <code>sort</code> , <code>direction</code> .
GET	/stats	Aggregated transaction statistics for the authenticated user.
PUT	/:id	Update a transaction by ID.
DELETE	/:id	Delete a single transaction.
DELETE	/bulk	Bulk delete transactions. Body: <code>ids</code> array.
DELETE	/all	Delete all transactions for the authenticated user.

4.3 Dashboard Endpoints

Base: /api/dashboard

GET	/	Full dashboard KPIs: revenue, expenses, profit, margin, growth, burn rate, runway, tax estimate, monthly breakdown, top categories.
-----	---	---

4.4 AI Endpoints

Chatbot: /api/chatbot

POST

/chat

Send a message to the AI financial assistant. Body: `message`, optional `sessionId`. Returns AI text response + usage stats.

GET

/history

Retrieve chat history for the current user.

DELETE

/history

Clear chat history.

AI Charts: /api/ai-charts

POST

/generate

Generate a chart from natural language prompt. Uses Groq to produce chart config + data. Limit-checked.

AI Excel: /api/ai-excel

POST

/generate

Generate and download an Excel file from a text description. Returns binary .xlsx file stream.

Gemini: /api/gemini

POST

/analyze

Analyze uploaded document using Google Gemini AI.

4.5 Subscription Endpoints

Base: /api/subscriptions

GET

/plans

List all available subscription plans with features and pricing. Public (no auth required).

GET

/current

Get the current user's subscription, plan details, and usage stats.

POST

/checkout

Subscribe to a plan. Body: `plan`, `billingCycle`, `cardNumber`, `expiryMonth`, `expiryYear`, `cvv`, `cardholderName`.

POST

/cancel

Cancel subscription (cancels at period end).

POST

/reactivate

Reactivate a subscription set to cancel at period end.

4.6 Public v1 API

API Key Auth — Base: /api/v1



Authentication

All v1 API endpoints (except `/ping`) require the header `X-API-Key: <your_api_key>`. Each endpoint also requires the key to have the appropriate scope.

Method	Endpoint	Scope Required	Description
GET	/ping	None	Health check — no auth
GET	/transactions	transactions:read	List transactions (paginated)
GET	/transactions/:id	transactions:read	Get single transaction
POST	/transactions	transactions:write	Create transaction
PUT	/transactions/:id	transactions:write	Update transaction
DELETE	/transactions/:id	transactions:write	Delete transaction
GET	/dashboard	dashboard:read	KPI summary



5. Frontend

5.1 Structure & Routing

The frontend is a React 19 SPA built with Vite 7. React Router v7 handles client-side routing. All authenticated routes are wrapped in `<ProtectedRoute>`.

Route Pattern	Component	Access
/	LandingPage	Public
/pricing	PricingPage	Public
/login	Login	Public (redirect if authed)
/register	Register	Public (redirect if authed)
/forgot-password	ForgotPassword	Public
/reset-password/:token	ResetPassword	Public
/app/:userId/dashboard	DashboardLayout	Protected
/app/:userId/transactions	TransactionsPage (lazy)	Protected
/app/:userId/ai-charts	AIChartsPage (lazy)	Protected
/app/:userId/excel	ExcelGenerator	Protected
/app/:userId/ai-excel	AIExcelGenerator	Protected
/app/:userId/chatbot	Chatbot	Protected
/app/:userId/excel-editor	ExcelUploadPage	Protected
/app/:userId/excel-editor/:fileId	ExcelEditorPage	Protected
/app/:userId/subscription	SubscriptionPage	Protected
/app/:userId/checkout	CheckoutPage	Protected
/app/:userId/developer	DeveloperSettingsPage (lazy)	Protected
/admin/*	AdminApp (lazy)	Admin only

5.2 Pages & Components

 **Dashboard**

KPI summary cards (revenue, expenses, profit, margin, growth), monthly revenue/expense bar charts, category breakdown pie chart, recent transactions list, and pending alerts panel. Uses Recharts for data visualization.

 **Transactions**

Full-featured transaction table with search, filter (type / category / status / date range), column sorting, inline editing, bulk select & delete, CSV export, and file upload with AI import wizard.

 AI Charts

Natural language chart generator. User types a prompt (e.g., "Show my expenses by category for Q1") and Groq AI generates an interactive Recharts visualization with a table view.

 AI Chatbot

Conversational financial assistant powered by Groq. Full chat history, markdown response rendering, usage meter showing remaining monthly AI messages.

 Excel Tools

Excel Editor: Upload, view, and edit Excel files in-browser. **AI Excel Generator:** Describe a spreadsheet in plain text and download the generated .xlsx file instantly.

 Landing Page

Marketing landing page with feature showcase, testimonials, pricing preview, and CTA buttons. Content managed via Admin CMS.

 Subscription

Current plan overview, usage meters (transactions, uploads, AI features), invoice history, plan upgrade/downgrade, and cancellation flow.

 Developer Settings

API key management: create named keys with specific scopes, view key prefix hints, revoke keys, and view usage documentation and code examples.

Admin Panel (AdminApp)

Accessible at `/admin` for admin users only. Includes:

- Platform stats dashboard (users, revenue, signups)
- User management table
- Blog post CMS editor
- Landing page content editor
- Subscription tier & pricing manager
- Navigation link editor

5.3 State Management

Layer	Tool	Responsibility
Authentication	React Context (<code>AuthContext</code>)	JWT token, userId, login/logout, persisted in localStorage
Theme	React Context (<code>ThemeContext</code>)	Dark / light mode toggle, persisted preference
Subscription	React Context (<code>SubscriptionContext</code>)	Current plan, usage counters, feature gate checks
Server State	TanStack Query v5	API data fetching, caching, background refetching, optimistic updates
UI / Local State	Zustand stores	UI preferences, filter state, editor state
Form State	React Hook Form + Zod	Form validation and submission handling



6. Subscription & Billing

The platform uses a tiered subscription model with simulated payment processing (no live payment gateway). Plans are dynamically configurable via the Admin tier manager.

Free	Pro	Business
\$0 / month	\$19 / month	\$49 / month
✓ 50 transactions / month	✓ 500 transactions / month	✓ Unlimited transactions
✓ 2 Excel file uploads	✓ 20 Excel file uploads	✓ Unlimited Excel uploads
✓ 10 AI chart generations	✓ 100 AI chart generations	✓ Unlimited AI chart generations
✓ 20 AI chat messages	✓ 500 AI chat messages	✓ Unlimited AI chat messages
✓ 5 AI Excel generations	✓ 50 AI Excel generations	✓ Unlimited AI Excel generations
✓ Basic dashboard	✓ Advanced dashboard	✓ Full dashboard + reporting
✓ Email support	✓ Priority email support	✓ Dedicated support
	✓ Export to PDF	✓ Custom integrations
	✓ Team collaboration (up to 3)	✓ Team collaboration (up to 10)

Annual Billing

Annual billing is available at **\$190/year** (Pro) and **\$490/year** (Business), equivalent to approximately 2 months free.

Usage Tracking

Usage is tracked per-user per-calendar-month via a JSON file store at `uploads/usage/store.json`. The `checkLimit` middleware enforces limits before creating new resources. Features tracked:

- `transactions` — manual transaction creations
- `excelUploads` — file upload operations
- `aiChartGenerations` — AI chart generate calls
- `aiChatMessages` — chatbot messages sent
- `aiExcelGenerations` — AI Excel generate calls

Test Payment Cards

**Simulated Payment Only**

No real payment gateway is connected. The checkout flow validates card format and uses predefined test card numbers to simulate success, decline, insufficient funds, and expiry scenarios. This is for demonstration purposes only.



7. Security

**Authentication**

- Stateless JWT tokens (72h expiry)
- bcrypt password hashing (10 salt rounds)
- Auth rate limiter: 10 attempts / 15 min in production
- Password reset tokens stored as SHA-256 hashes only
- Reset tokens expire after 1 hour
- isAdmin flag embedded in JWT at sign time

**HTTP Security**

- Helmet sets: CSP, HSTS, X-Frame-Options, X-Content-Type-Options
- CORS: production restricts to explicit origin whitelist
- Global rate limiter: 2000 req / 15 min per IP
- Trust proxy enabled for correct IP with Nginx / Render
- Request body size limit: 25 MB

**API Keys**

- Zero-knowledge storage: only SHA-256(key) stored
- Constant-time comparison to prevent timing attacks
- Scope-based access control on every v1 endpoint
- Keys can be revoked (isActive = false)
- Optional expiry dates
- Last-used timestamp tracking

**Input Validation**

- All auth endpoints use express-validator rules
- Transaction query params validated with Zod schema
- File uploads restricted to .xlsx, .xls, .pdf (type + extension)
- Max file size enforced at Multer level (10 MB)
- Mongoose schemas provide secondary type enforcement

Error Handling

- Centralized error middleware — no raw errors leak to client
- Stack traces hidden in production
- Request IDs for tracing errors across logs
- Unhandled promise rejections caught globally

Data Security

- Card numbers are never stored — only brand, last4, expiry
- Admin emails never appear in user-facing lists
- File access is scoped by authenticated user ID
- MongoDB connection with Mongoose strict schema enforcement



8. Developer Guide

8.1 Local Setup

Prerequisites

- Node.js 20+ and npm 10+
- MongoDB (local or Atlas URI)
- Groq API key (free at console.groq.com)
- Google Generative AI API key (optional — for Gemini features)
- Resend or SMTP credentials (optional — for email features)

Installation

```
# 1. Clone and install root dependencies
npm install

# 2. Install backend dependencies
cd backend
npm install

# 3. Install frontend dependencies
cd ../frontend
npm install

# 4. Create backend environment file
cp backend/.env.example backend/.env
# Edit .env with your values

# 5. Start backend server (port 5000)
cd backend
node server.js

# 6. Start frontend dev server (port 5173)
cd ../frontend
npm run dev
```

**MongoDB is Optional in Development**

The app automatically detects when MongoDB is unavailable and switches to a local file-based data store. All features work without a database during local development.

8.2 Environment Variables

Create a `.env` file in the `backend/` directory:

Variable	Required	Description
PORT	Optional	Server port (default: 5000)
NODE_ENV	Optional	Set to <code>production</code> to enable rate limiting, hide stack traces
JWT_SECRET	Required	Long random string for JWT signing. Use dev fallback in development.
MONGODB_URI	Optional*	MongoDB connection string. App uses local store if absent.
CLIENT_URL	Optional*	Comma-separated allowed CORS origins (production)
GROQ_API_KEY	Optional*	Groq API key. Required for chatbot, AI charts, AI Excel features.
GEMINI_API_KEY	Optional*	Google Gemini API key. Required for document AI analysis.
ADMIN_EMAILS	Optional	Comma-separated admin email addresses
EMAIL_FROM	Optional	Sender address for transactional emails
RESEND_API_KEY	Optional	Resend API key (preferred email provider)
SMTP_HOST	Optional	SMTP host for Nodemailer fallback
SMTP_PORT	Optional	SMTP port (typically 587 or 465)
SMTP_USER	Optional	SMTP username
SMTP_PASS	Optional	SMTP password
MAX_FILE_SIZE	Optional	Max upload size in bytes (default: 10 MB)

8.3 Developer API Keys

Users can generate API keys from the **Developer Settings** page (`/app/:userId/developer`).

Creating an API Key (via UI)

1. Navigate to Developer Settings
2. Click "Create New API Key"
3. Enter a name and select scopes
4. Copy the full key immediately — it will only be shown once

Using the API Key

```
# List transactions
curl https://your-api.com/api/v1/transactions \
  -H "X-API-Key: ak_your_key_here"

# Create a transaction
curl -X POST https://your-api.com/api/v1/transactions \
  -H "X-API-Key: ak_your_key_here" \
  -H "Content-Type: application/json" \
  -d '{"date":"2026-06-01","desc":"Office rent","amount":1200,"type":"expense","category":

# Get dashboard KPIs
curl https://your-api.com/api/v1/dashboard \
  -H "X-API-Key: ak_your_key_here"
```

Available Scopes

Scope	Access
<code>transactions:read</code>	List and retrieve transactions
<code>transactions:write</code>	Create, update, delete transactions
<code>dashboard:read</code>	Read KPI dashboard data
<code>excel:read</code>	Download Excel files
<code>excel:write</code>	Upload and modify Excel files
<code>chatbot:read</code>	Access chatbot and chat history



9. Deployment

9.1 Backend Deployment (Render / Railway / Fly.io)

1. Set all required environment variables in the platform dashboard
2. Set `NODE_ENV=production`
3. Start command: `node server.js` from the `backend/` directory
4. Set `CLIENT_URL` to your frontend domain
5. The `/health` endpoint serves as the health check URL

9.2 Frontend Deployment (Vercel / Netlify / GitHub Pages)

- 1. Set the build command to `npm run build` in the `frontend/` directory
- 2. Set the publish directory to `frontend/dist`
- 3. Create `frontend/.env.production` with: `VITE_API_URL=https://your-backend-url.com`
- 4. Configure SPA fallback: all routes → `index.html`

9.3 Production Checklist

Item	Action
JWT Secret	Generate a strong random secret (64+ chars) — do not use the dev fallback
MongoDB URI	Use MongoDB Atlas with TLS enabled; restrict IP allowlist
CORS Origin	Set <code>CLIENT_URL</code> to your exact frontend domain (no trailing slash)
Rate Limiting	Enabled automatically when <code>NODE_ENV=production</code>
Email Service	Configure Resend API key or SMTP credentials for password reset emails
Admin Emails	Set <code>ADMIN_EMAILS</code> to restrict admin panel access
File Storage	Consider S3 or similar for uploaded files in multi-instance deployments



Local File Stores in Production

The local JSON-based data stores (usage, subscriptions, auth fallback) write to the server filesystem. These are not suitable for multi-instance deployments. Ensure MongoDB is connected in production so all data uses the database.



10. Email Service

Email is handled by `backend/services/emailService.js` which supports two providers with automatic fallback:

 Resend (Primary)

Modern email API. Configure with `RESEND_API_KEY`. Preferred for production use. Supports custom domains with SPF/DKIM configuration.

 Nodemailer (Fallback)

SMTP-based email. Configure with `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`. Works with Gmail, SendGrid, Mailgun, etc.

Triggered Emails

Event	Template	Notes
Password Reset Request	HTML email with reset link	Token expires in 1 hour. Link format: <code>/reset-password/:token</code>

Development Mode

 Console Fallback in Development

When no email provider is configured, the email service logs the reset token directly to the server console. This allows testing the password reset flow without email credentials.



11. Conclusion & Future Work

11.1 Conclusion

This graduation project has demonstrated the technical feasibility and practical value of integrating large language model (LLM) capabilities within a conventional financial management software architecture. The AI Accountant system successfully achieves all primary research objectives: it provides SME operators with real-time financial dashboards and KPI analytics, enables AI-driven natural-language querying via a conversational chatbot interface, automates transaction extraction from uploaded documents using Google Gemini's multi-modal analysis capabilities, and generates fully formatted spreadsheets from plain-text descriptions using the Groq LLaMA 3.3-70B inference engine.

The implementation adheres to contemporary software engineering best practices, including layered RESTful API design, stateless JWT-based authentication, scope-controlled API key authorization, bcrypt password hashing, and a defense-in-depth HTTP security model enforced through the Helmet middleware suite. The dual data-store architecture — which transparently falls back to a local JSON-based persistence layer in the

absence of a live MongoDB connection — substantially reduces the barrier to entry for local development and testing workflows, representing a pragmatic engineering contribution.

The tiered subscription model demonstrates a commercially viable pathway for platform monetization, while the versioned public REST API (v1) with scope-based API key authentication enables programmatic third-party developer integration. Collectively, this system constitutes a meaningful contribution at the intersection of enterprise web engineering and applied artificial intelligence, particularly within the domain of SME financial management.

11.2 Limitations

⚠ Simulated Payment Processing

The subscription billing module employs a simulated payment flow rather than a live payment gateway integration. Real-world deployment would necessitate integration with a PCI-DSS-compliant payment processor such as Stripe or PayPal, and would require handling recurring billing, refunds, and dispute resolution.

⚠ Local File Storage

Uploaded files and the JSON-based usage store are persisted on the local server filesystem, which is incompatible with horizontally scaled, multi-instance deployment environments. A distributed object storage solution (e.g., Amazon S3, Google Cloud Storage) would be required for production-scale, high-availability deployments.

⚠ External AI Dependency

All AI feature responses are contingent on the availability of external API services from Groq and Google Gemini, and are subject to third-party rate limits, inference latency, and pricing changes. High-concurrency scenarios may experience elevated response times on AI-powered endpoints.

⚠ Single-Tenant Data Model

The current data model is designed for single-user tenancy, with data isolation enforced by authenticated user ID. Support for organizational accounts, multi-user collaborative teams, or enterprise multi-tenancy would require substantial schema redesign and role-based access control enhancements.

11.3 Future Work

The following directions are identified as high-priority areas for future research and development:

- **Live Payment Gateway Integration:** Replacing the simulated checkout flow with a production-grade, PCI-DSS-compliant payment processor to enable real subscription revenue collection and automated recurring billing management.
- **Automated Tax Reporting:** Extending the KPI analytics engine to generate jurisdiction-specific tax reports and integrate with government electronic filing APIs (e.g., VAT returns, income tax declarations).
- **OCR Enhancement for Scanned Documents:** Implementing Optical Character Recognition (OCR) post-processing to improve transaction extraction accuracy from scanned or image-based PDF financial documents where embedded text is unavailable.
- **Multi-Currency & Localization Support:** Incorporating real-time foreign exchange rate feeds and locale-aware number formatting to support multi-currency transaction recording and consolidated international financial reporting.

- **Mobile Application Development:** Developing a cross-platform companion mobile application (React Native or Flutter) to extend platform accessibility to mobile business operators and enable push-notification financial alerts.
- **Predictive Financial Analytics:** Applying time-series forecasting models (e.g., ARIMA, LSTM neural networks) to historical transaction data in order to project future cash flow, revenue trends, and expense trajectories.
- **Domain-Specific Model Fine-Tuning:** Fine-tuning a compact language model on accounting and financial domain corpora to reduce dependency on general-purpose third-party API services and improve the accuracy of domain-specific financial reasoning and categorization.
- **Comprehensive Audit Trail & Regulatory Compliance:** Implementing a tamper-evident, append-only audit log for all financial record modifications to satisfy regulatory compliance requirements such as Sarbanes-Oxley (SOX) and International Financial Reporting Standards (IFRS).



12. References & Bibliography

12.1 Frameworks & Libraries

1. Meta Platforms Inc. *React — A JavaScript library for building user interfaces*. Version 19.0. [Online]. Available: <https://react.dev> [Accessed June 2026].
2. OpenJS Foundation. *Node.js — JavaScript runtime built on Chrome's V8 JavaScript engine*. Version 20 LTS. [Online]. Available: <https://nodejs.org> [Accessed June 2026].
3. T. Holowaychuk. *Express — Fast, unopinionated, minimalist web framework for Node.js*. Version 5.0. [Online]. Available: <https://expressjs.com> [Accessed June 2026].
4. MongoDB Inc. *MongoDB — The Developer Data Platform*. [Online]. Available: <https://www.mongodb.com> [Accessed June 2026].
5. Automattic Inc. *Mongoose — Elegant MongoDB Object Modeling for Node.js*. Version 8. [Online]. Available: <https://mongoosejs.com> [Accessed June 2026].
6. E. You. *Vite — Next Generation Frontend Tooling*. Version 7.0. [Online]. Available: <https://vitejs.dev> [Accessed June 2026].
7. A. Wathan. *Tailwind CSS — A Utility-First CSS Framework for Rapid UI Development*. Version 4. [Online]. Available: <https://tailwindcss.com> [Accessed June 2026].
8. Recharts Team. *Recharts — Redefined Chart Library Built with React and D3*. [Online]. Available: <https://recharts.org> [Accessed June 2026].
9. ExcelJS Contributors. *ExcelJS — Excel Workbook Manager for Node.js*. [Online]. Available: <https://github.com/exceljs/exceljs> [Accessed June 2026].
10. Auth0 Inc. *node-jsonwebtoken — JSON Web Token Implementation for Node.js*. [Online]. Available: <https://github.com/auth0/node-jsonwebtoken> [Accessed June 2026].
11. TanStack. *TanStack Query (React Query) — Powerful Asynchronous State Management for TypeScript/JavaScript*. Version 5. [Online]. Available: <https://tanstack.com/query> [Accessed June 2026].

12. P. Krzysztof and contributors. *Zustand — A Small, Fast, and Scalable State Management Solution for React*. [Online]. Available: <https://github.com/pmndrs/zustand> [Accessed June 2026].

12.2 AI Services & Academic References

13. Groq Inc. *Groq API Documentation — LLaMA 3.3-70B Versatile Model Reference*. [Online]. Available: <https://console.groq.com/docs> [Accessed June 2026].
14. Meta AI Research. *The Llama 3 Herd of Models*. arXiv preprint arXiv:2407.21783, 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>
15. Google DeepMind. *Gemini: A Family of Highly Capable Multimodal Models*. arXiv preprint arXiv:2312.11805, 2023. [Online]. Available: <https://arxiv.org/abs/2312.11805>
16. Google Inc. *Google AI for Developers — Gemini API Documentation*. [Online]. Available: <https://ai.google.dev> [Accessed June 2026].
17. T. Brown, B. Mann, N. Ryder et al. *Language Models are Few-Shot Learners*. Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 1877–1901, 2020.
18. A. Vaswani, N. Shazeer, N. Parmar et al. *Attention Is All You Need*. Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

12.3 Security Standards & Protocols

19. OWASP Foundation. *OWASP Top Ten — The Ten Most Critical Web Application Security Risks*. 2021 Edition. [Online]. Available: <https://owasp.org/www-project-top-ten> [Accessed June 2026].
20. M. Jones, J. Bradley, and N. Sakimura. *RFC 7519: JSON Web Token (JWT)*. Internet Engineering Task Force (IETF), May 2015. [Online]. Available: <https://tools.ietf.org/html/rfc7519>
21. N. Provos and D. Mazières. *A Future-Adaptable Password Scheme*. Proceedings of the USENIX Annual Technical Conference (FREENIX Track), 1999, pp. 32–32.
22. R. T. Fielding. *Architectural Styles and the Design of Network-Based Software Architectures*. Ph.D. Dissertation, University of California, Irvine, 2000.
23. D. Hardt. *RFC 6749: The OAuth 2.0 Authorization Framework*. Internet Engineering Task Force (IETF), October 2012. [Online]. Available: <https://tools.ietf.org/html/rfc6749>

12.4 Related Systems & Comparative Platforms

24. Intuit Inc. *QuickBooks Online — Cloud-Based Small Business Accounting Software*. [Online]. Available: <https://quickbooks.intuit.com> [Accessed June 2026].
25. Xero Ltd. *Xero — Online Accounting Software for Small Business*. [Online]. Available: <https://www.xero.com> [Accessed June 2026].
26. Wave Financial Inc. *Wave — Free Accounting and Invoicing Software for Small Business*. [Online]. Available: <https://www.waveapps.com> [Accessed June 2026].